

HDOC – Day 32

C Programming Assignment

Date-25/09/26

Problem Statement

Write a C program to perform the following menu-driven operations using separate functions: (1) get two variable-length 2D arrays and prove that matrix addition is commutative, (2) get two variable-length 2D arrays and prove that matrix multiplication is not commutative, and (3) exit.

Concept

For addition, matrices A and B of the same order satisfy $A + B = B + A$. For multiplication, in general $A \times B$ is not equal to $B \times A$. The program uses C99 variable-length arrays (VLAs) and separate functions for input, display, addition, multiplication, comparison, and the two menu operations.

Algorithm

- 1 Display the menu and read the user's choice.
- 2 For addition, read rows and columns, then input matrices A and B.
- 3 Compute $A+B$ and $B+A$; compare corresponding elements and display both results.
- 4 If all corresponding elements are equal, state that matrix addition is commutative.
- 5 For multiplication, read order n and input two $n \times n$ matrices A and B.
- 6 Compute $A \times B$ and $B \times A$ using three nested loops; display both products.
- 7 Compare the products. If they differ, the entered matrices demonstrate non-commutativity; if equal, explain that this particular case is commutative.
- 8 Repeat until choice 3 is selected.

C Program

```
#include <stdio.h>

void inputMatrix(int r, int c, int a[r][c], const char *name) {
    int i, j;
    printf("Enter elements of matrix %s:\n", name);
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
}

void displayMatrix(int r, int c, int a[r][c]) {
    int i, j;
    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++)
            printf("%d\t", a[i][j]);
        printf("\n");
    }
}

void addMatrix(int r, int c, int a[r][c], int b[r][c],
               int sum[r][c]) {
    int i, j;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            sum[i][j] = a[i][j] + b[i][j];
}

int equalMatrix(int r, int c, int a[r][c], int b[r][c]) {
    int i, j;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            if (a[i][j] != b[i][j])
                return 0;
    return 1;
}

void multiplyMatrix(int n, int a[n][n], int b[n][n],
```

```

int product[n][n] {
int i, j, k;
for (i = 0; i < n; i++) {
for (j = 0; j < n; j++) {
product[i][j] = 0;
for (k = 0; k < n; k++)
product[i][j] += a[i][k] * b[k][j];
}
}
}

void proveAdditionCommutative(void) {
int r, c;
printf("Enter number of rows and columns: ");
scanf("%d %d", &r, &c);
if (r <= 0 || c <= 0) {
printf("Invalid matrix dimensions.\n");
return;
}
int A[r][c], B[r][c], AB[r][c], BA[r][c];
inputMatrix(r, c, A, "A");
inputMatrix(r, c, B, "B");
addMatrix(r, c, A, B, AB);
addMatrix(r, c, B, A, BA);
printf("\nA + B:\n");
displayMatrix(r, c, AB);
printf("\nB + A:\n");
displayMatrix(r, c, BA);
if (equalMatrix(r, c, AB, BA))
printf("\nA + B = B + A\nMatrix addition is commutative.\n");
}

void proveMultiplicationNotCommutative(void) {
int n;
printf("Enter order of the square matrices: ");
scanf("%d", &n);
if (n <= 0) {

```

```

printf("Invalid matrix order.\n");
return;
}
int A[n][n], B[n][n], AB[n][n], BA[n][n];
inputMatrix(n, n, A, "A");
inputMatrix(n, n, B, "B");
multiplyMatrix(n, A, B, AB);
multiplyMatrix(n, B, A, BA);
printf("\nA x B:\n");
displayMatrix(n, n, AB);
printf("\nB x A:\n");
displayMatrix(n, n, BA);
if (!equalMatrix(n, n, AB, BA))
printf("\nA x B != B x A\n"
"Hence, matrix multiplication is not commutative.\n");
else
printf("\nFor these particular matrices, A x B = B x A.\n"
"Choose another pair to demonstrate non-commutativity.\n");
}
int main(void) {
int choice;
do {
printf("\n--- MATRIX OPERATIONS MENU ---\n");
printf("1. Prove matrix addition is commutative\n");
printf("2. Prove matrix multiplication is not commutative\n");
printf("3. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice) {
case 1:
proveAdditionCommutative();
break;
case 2:
proveMultiplicationNotCommutative();
break;

```

```
case 3:
printf("Exiting program.\n");
break;
default:
printf("Invalid choice. Please enter 1, 2 or 3.\n");
}
} while (choice != 3);
return 0;
}
```